



Universidad
Indoamérica

ユニット 7・オペレーティングシステムの基礎

スレッドと並行性

Universidad Tecnológica Indoamérica 演習 TD6

スレッド：プロセスの中の逐次的な実行

- 「軽量プロセス」とも：1つのプロセスの中の複数の実行
- すべてが**同じアドレス空間**を共有する
- スレッドの間には**メモリ保護がない**
- プロセス = 仮想メモリ（保護） + スレッド（並列実行）

何が共有されるか、実際のアドレスで

プロセス・8 thread(s)

コード

データ : shared

0x650ff476602c = 8

heap

ディスクリプタ 012

全スレッドで共有

スレッド1

スタック

local 0x31fee74

レジスタ : PC、SP...

スレッド2

スタック

local 0x19fbe74

レジスタ : PC、SP...

スレッド3

スタック

local 0xbffee74

レジスタ : PC、SP...

スレッド4

スタック

local 0x29fde74

レジスタ : PC、SP...

スレッド5

スタック

local 0x21fce74

レジスタ : PC、SP...

スレッド6

スタック

local 0x11fae74

レジスタ : PC、SP...

スレッド7

スタック

local 0x09f9e74

レジスタ : PC、SP...

スレッド8

スタック

local 0xb7fde74

レジスタ : PC、SP...

スタックの間隔 8.004 MiB (8 MiB + 4 KiB のガード)

マルチスレッドのプログラム

- 組み込みシステム：1つのプログラム、並行する操作
- カーネル自身：何千ものカーネルスレッド
- データベースとネットワークサーバ（ユニット 8）
- 並列プログラム：OpenMP、コアごとに1スレッド

現在はさらに

ブラウザ（タブごと・タスクごとのスレッド）、ゲームエンジン、CPU での言語モデルの推論（ユニット 9）、そしてバージョン 3.13 からグローバルロック（GIL）なしの版がある Python。

スレッドの作成は安い...どれくらい？

実際の出力

```
$ ./create_cost 2000
2000 threads:      41.2 us per pthread_create + pthread_join
2000 processes:   248.8 us per fork + waitpid
a process costs 6.0 times a thread
```

- コピーするページはない：新しいスタックとタスク構造体だけ
- システムはスレッドを1つずつスケジュールする：**実行順序を仮定しない**

スレッドの作成、終了、待機

C

```
int pthread_create(pthread_t *t, const pthread_attr_t *attr, /* NULL: defaults */
                  void *(*start)(void *), void *arg);
void pthread_exit(void *retval); /* = return retval from start; never returns */
int pthread_join(pthread_t t, void **retval); /* PTHREAD_CANCELED if t was cancelled */
int pthread_detach(pthread_t t); /* nobody will join t: freed when it ends */
```

- `attr` では、たとえば *joinable* (既定) か *detached* を選ぶ: `pthread_attr_init` と `pthread_attr_setdetachstate` で設定する
- 終了したスレッドは、誰かが `join` するまで (または *detached* でない限り) スタックと状態を**解放しない**: さもないとゾンビのようなメモリリークになる
- `join` は `t` が終わるまで呼び出し側を止め、その `retval` を受け取る。 *detached* なスレッドは `join` できない
- Linux では `pthread_create` は `clone` 呼び出しを使う: 同じアドレス空間、同じディスクリプタ

pthread_create、pthread_exit、pthread_join

C tds/td6/threads.c

```
for (int i = 0; i < NTHREADS; i++) {
    int rc = pthread_create(&tids[i], NULL, start, (void *)(&i));
    exit_if(rc != 0, "pthread_create"); /* pthread functions return the error code */
}
for (int i = 0; i < NTHREADS; i++)
    exit_if(pthread_join(tids[i], NULL) != 0, "pthread_join");
```

実際の出力

```
$ ./threads
thread 57632 of process 357789: shared[0x650ff476602c] = 1, local[0x7626e31fee74] = 1
thread 79520 of process 357789: shared[0x650ff476602c] = 4, local[0x7626e19fbe74] = 1
...
```

修正（スライド 137～140）：`-lthread` ではなく `-pthread`。`pthread_*` は 0 かエラー番号を返し、`errno` と -1 は返さない。

スレッドとシグナル

シグナルは**プロセス全体**のもの。スレッドについては：

プロセス	スレッド
<code>kill()</code>	<code>pthread_kill()</code>
<code>sigprocmask()</code>	<code>pthread_sigmask()</code>
<code>sigsuspend()</code>	<code>sigwait()</code> (シグナルを待つ専用のスレッド)

並行性と共有メモリ：頭痛の種

たいていスレッドは別々のデータを触り、順序は関係ない。だが、最初に `x = 1` なら：

C

```
/* thread 1 */      /* thread 2 */  
x = x + 2;           x = x * 3;
```

`x` はどんな値になりうる？ → **非決定的**。

- スレッドにも、共有メモリを使うプロセス（ユニット 6）にも同じく当てはまる
- 再現しにくいバグ：何百万回に1回しか現れない
- もっと悪い例（スライド 146）：
 `while (x < 1000) x++;` 対
 `while (x > -1000) x--;` どちらが勝つこともありうる...永遠に終わらないことも

20 通りのインターリーブを1つつ

スレッド 1: $x = x + 2$;

1. $r1 = x$

2. $r1 = r1 + 2$

6. $x = r1$

スレッド 2: $x = x * 3$;

3. $r2 = x$

4. $r2 = r2 * 3$

5. $x = r2$

メモリ

$x=1$

$x=1$

$x=1$

$x=1$

$x=3$

$x=3$

20 通りのインターリーブの結果

$x = 3: 18$

$x = 9: 1$

$x = 5: 1$

実測、TD6 (8 スレッド、チケット 100,000 枚):

`tickets_race`: 103,195 販売数 (`tickets = -7`)

`tickets_mutex`: 100,000 販売数 (`tickets = 0`)

`tickets_sem`: 100,000 販売数 (`tickets = 0`)

アトミック操作

- 完全に実行されるか、まったく実行されないか：
分割できない
- 典型例：ネイティブなサイズのメモリアクセス、
単純な命令
- アトミックでない：`x++`（読む・足す・書く）、32
ビットマシンでの 64 ビットアクセス

現代の C では

C11 以降、2 つのスレッドが同期なしに同じ
変数にアクセスする（少なくとも 1 つが
書き込み）のは**データ競合：未定義動作**。
アトミックには `_Atomic`、`<stdatomic.h>`、
`__atomic_add_fetch`（ユニット 6 の
`shm_fork` が使う）。`volatile` は何も
アトミックにしない。

牛乳が多すぎる：ソフトウェアによる相互排除

スレッド1	スレッド2
冷蔵庫を見る：牛乳がない	
店に行く	冷蔵庫を見る：牛乳がない
牛乳を買う	店に行く
帰ってくる	牛乳を買う...多すぎ！

- **同期**：アトミック操作を使ってスレッドを協調させる
- **相互排除**：一度に1つのスレッドだけがその動作を行う
- **クリティカルセクション**：一度に1つのスレッドしか実行できないコード
- 解1～3：冷蔵庫に鍵をかける、メモ1枚、スレッドごとにメモ1枚...そしてその失敗

Peterson と Dekker : 正しい...紙の上では

C

```

/* Peterson, thread 1 */
postit_1 = true; turn = 2;
while (postit_2 && turn == 2)
    ;                               /* busy waiting */
if (!milk) milk++;                 /* critical section */
postit_1 = false;

```

- メモリアクセスがアトミックで**順序どおり**なら正しい
- だがコンパイラとプロセッサは**並べ替える** (スライド 154)
- しかもビジーウェイトする : CPU を浪費する

実際の出力

```

$ for i in 1 2 3 4 5; do ./peterson; done; for i in 1 2 3 4 5; do ./peterson_fence; done
Peterson no fence : counter = 4000000, expected 4000000, lost 0
Peterson no fence : counter = 4000000, expected 4000000, lost 0
Peterson no fence : counter = 4000000, expected 4000000, lost 0
...
Peterson with fence: counter = 4000000, expected 4000000, lost 0
Peterson with fence: counter = 4000000, expected 4000000, lost 0
...

```

すべての解、すべての状態

スレッド 0

```
postit_0 = 1
turn = 1
/* フェンスなし */
loop: q = postit_1
if (!q) goto cs
t = turn
if (t ≠ 1) goto cs
goto loop
cs: enter()
r = milk
if (r) goto x
milk++
x: leave()
postit_0 = 0
end
```

スレッド 1

```
postit_1 = 1
turn = 0
/* フェンスなし */
loop: q = postit_0
if (!q) goto cs
t = turn
if (t ≠ 0) goto cs
goto loop
cs: enter()
r = milk
if (r) goto x
milk++
x: leave()
postit_1 = 0
end
```

共有メモリ

```
milk = 0
postit_0 = 0
postit_1 = 0
turn = 0
```

スレッド 0 buffer:p0=1, turn=1

スレッド 1 buffer:p1=1, turn=0

両方がクリティカルセクションに！

網羅的な探索

牛乳 1 リットルで終わる：最終状態 14 個

牛乳 2 リットルで終わる：最終状態 4 個

最短の反例：12 手順（手順 12）

アトミック命令：test-and-set と compare-and-swap

c

```
int test_and_set(int *p) { /* atomic */
    int old = *p; *p = 1; return old;
}
/* spinlock: busy-waits until the old value is 0 */
void lock(int *l) { while (test_and_set(l)) ; }
void unlock(int *l) { *l = 0; }
```

- x86 : `xchg`、`lock cmpxchg`、`lock xadd`。
ARM64 : `ldxr` / `stxr` か `cas`
- ビジーウェイトは CPU を無駄にし、**優先度逆転**を起こしうる
- 待たないロック？ 無理。
ビジーウェイトなしなら？ 一部は →

限定的なビジーウェイト、今日の形：futex

実際の出力

```
$ strace -f -c -e trace=futex ./lock_cost 1 100000; strace -f -c -e trace=futex ./lock_cost 8 100000
1 thread(s) x 100000 lock/unlock: counter = 100000, 48.4 ns per lock/unlock
  0.00    0.000000    0    1    futex
8 thread(s) x 100000 lock/unlock: counter = 800000, 85.5 ns per lock/unlock
100.00    0.223730    27    8013    3116 futex
```

- 空きミューテックス → ユーザ空間でアトミック命令1つ、**システムコールなし**
- 使用中のミューテックス → `futex(WAIT)` : カーネルがスレッドを眠らせ、`unlock` で起こす
- `pthread_spin_lock` は決して眠らない (ビジーウェイト) : 眠るほうが回るより高くつく、とても短い区間だけに使う

ロック： `pthread_mutex_lock` / `unlock`

C tds/td6/tickets_mutex.c

```
static pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
...
for (;;) {
    pthread_mutex_lock(&lock);
    if (tickets ≤ 0) {                /* checked w.
        pthread_mutex_unlock(&lock);
        break;
    }
    sold++;
    sched_yield();
    tickets--;
    pthread_mutex_unlock(&lock);
}
```

または動的に： `pthread_mutex_init(&m, attr)`

(`pthread_mutexattr_t`、 `NULL` = 既定値) ...

`pthread_mutex_destroy(&m)`。

実際の出力

```
$ ./tickets_race 100000 | tail -3
...
103195 tickets sold out of 100000 (tickets = -7)
```

実際の出力

```
$ ./tickets_mutex 100000 | tail -3
...
100000 tickets sold out of 100000 (tickets = 0)
```

修正 (スライド 162)：返すのは 0 かエラー番号
(`EDEADLK` はエラー検査付きミューテックスのみ)
で、-1 ではない。

セマフォ (Dijkstra、1960 年代末)

2 つのアトミック操作を持つ 0 以上の整数：**P** (*proberen* : 正になるまで待つ 1 引く) と **V** (*verhogen* : 1 足して起こす)。

- 初期値 1 → ロック
- 初期値 0 → 別のスレッドを待つ
- 初期値 k → 同時に最大 k 個

C

```
int sem_init(sem_t *s, int pshared, unsigned value);
int sem_destroy(sem_t *s); /* unnamed */
sem_t *sem_open(const char *name, int flags /* O_CREAT [| O_EXCL] */,
                mode_t mode, unsigned value); /* SEM_FAILED */
int sem_close(sem_t *s); /* named: this process */
int sem_unlink(const char *name); /* named: the name */
int sem_wait(sem_t *s); /* P */
int sem_post(sem_t *s); /* V */
```

修正 (スライド 167) : `sem_open` は (-1 ではなく)

`SEM_FAILED` を返し、既定値はない : `O_CREAT` では `value` を渡す必要がある。また `sem_close` はセマフォを削除しない : 名前は `sem_unlink` まで `/dev/shm` に残る。

生産者と消費者の問題

- 生産者は大きさに限りのある共有バッファにデータを書く
- 消費者がそれを取り出す
- 生産者はバッファが満杯なら待ち、消費者は空なら待つ
- バッファは変更される：クリティカルセクションが必要

例：コマンドのパイプライン（ユニット 5）、
`gcc (cpp | cc1 | as | ld)`、
画像フィルタ、サービス間のメッセージキュー。

実際の出力

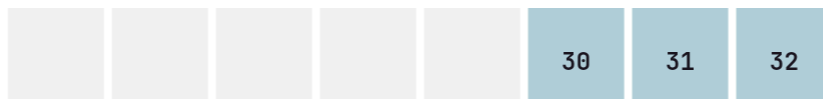
```
$ ./prodcons_multi 4 100000  
consumer 2: 23918 items  
consumer 3: 26423 items  
consumer 1: 23037 items  
consumer 0: 26622 items  
sum = 5000050000, expected 5000050000: OK
```

2つのセマフォと1つのミューテックスを一步步

生産者

```
in = (in+1) % BUF
v = 32・稼働中
```

バッファ (BUF = 8)



out

in

empty = 5 full = 0 out_lock: C2

消費者 1・8

```
wait(full)
待機: full = 0
```

消費者 2・5

```
unlock(out_lock)
稼働中
```

消費者 3・9

```
wait(full)
待機: full = 0
```

消費者 4・7

```
lock(out_lock)
ミューテックス待ち
```

40 個の値: 緑 = 1 回・赤 = 消失・橙 = 重複



実測、prodcons_multi 4 100000 : 消費者ごとの値の数

C1: 26,622

C2: 23,037

C3: 23,918

C4: 26,423

条件変数

C

```
pthread_mutex_lock(&mutex);
while (!condition)           /* while, not if */
    pthread_cond_wait(&cond, &mutex);
/* condition is true, mutex held */
pthread_mutex_unlock(&mutex);
/* whoever changes the condition: */
pthread_cond_signal(&cond);   /* wakes one */
pthread_cond_broadcast(&cond); /* wakes all */
```

- (カウンタに限らず) どんな条件でも待つ
- `cond_wait` はミューテックスを放して眠るのを **アトミック**に行い、起きたときに取り直す
- `if` ではなく `while` : 偽の起床があったり、別のスレッドが条件を変えていたりする

デッドロック

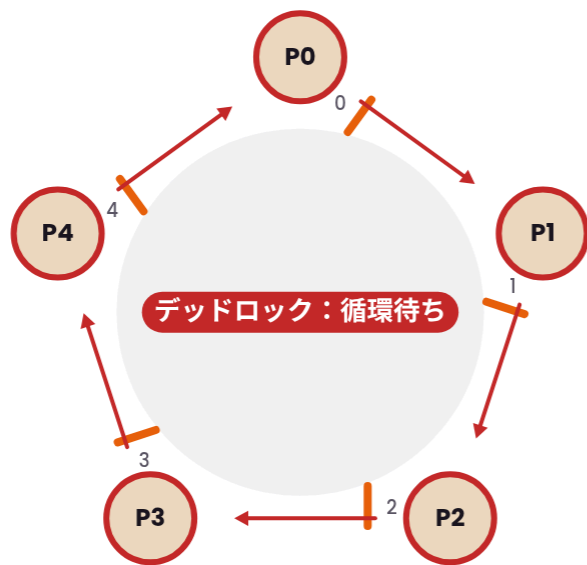
C

```
/* thread 1 */      /* thread 2 */
x.P();              y.P();
y.P();              x.P(); /* ← may never return */
/* ... */           /* ... */
y.V(); x.V();        x.V(); y.V();
```

いつも起きるわけではない：実行次第。複数の資源が必要で、外からの介入なしには解決しない。

- 資源：CPU、メモリ、ディスク、デバイス。
共有できるかどうか、横取りできるかどうか
- 例：優先権のある交差点で、各車が1区間を占めて次の区間を待つ
- 抜け出し方：巻き戻す、資源を取り上げる
(横取り) ...飢餓の危険を伴う

食事する哲学者



■ 考え中

■ 空腹

■ 食事中

■ 完了

→ 待っている (グラフ)

Coffman の条件

- ✓ 相互排除
- ✓ 保持と待機
- ✓ 横取り不可
- ✓ 循環待ち

4つの条件とその防ぎ方

1. 相互排除
 2. 他の資源を待ちながら資源を保持する
 3. 横取り不可
 4. 循環待ち： T_1 が T_2 を待ち、...、 T_n が T_1 を待つ
- 取得の**全体順序**（4 を崩す）：TD6 の修正
 - 横取りする（3 を崩す）：状態が不整合になる危険あり
 - 割り当てグラフの循環を検出する。銀行家のアルゴリズムで避ける
 - タイムアウト（`trylock`、`timedlock`）：保証はない

まとめ

- 問題：非決定的な結果、デッドロック、飢餓
- 共有資源への書き込みはすべて → クリティカルセクションの中で
- ソフトウェアによる相互排除は紙の上では動く。実際のハードウェアでは
ミューテックス、セマフォ、アトミック操作
- デッドロックは再現しにくく、4つの条件がすべてそろう必要がある

解答付き TD6 + AI チャレンジ

- `threads`、`tickets_race/mutex/sem`
- `prodcons`、`prodcons_multi`、`philosophers`（とデッドロックする版）
- 新規：`create_cost`、`peterston`、`lock_cost`

Bash

```
make -C tds && tds/tests/run.sh td6
```



Universidad
Indoamérica

まとめ

3つの問い

1. なぜ `tickets_race` は実際の枚数より多くのチケットを売るのか？
2. Peterson はなぜ x86 で失敗し、何で直るのか？
3. 箸の全体順序はどの条件を崩すのか？